

Detailed Line-by-Line Explanation of OpenMP Sorting Code

This document provides a detailed explanation of every important line in the uploaded C++ program. The code demonstrates: Sequential Bubble Sort Parallel Bubble Sort using OpenMP Sequential Merge Sort Parallel Merge Sort using OpenMP Tasks

Line	Code	Detailed Explanation
1	#include <iostream>	Includes the Input/Output stream library. This allows the program to use cout for output and cin for input.
2	#include <vector>	Includes the vector container library from STL. vector is a dynamic array that can resize automatically.
3	#include <omp.h>	Includes the OpenMP library used for parallel programming and thread management.
4	#include <cstdlib>	Includes standard utility functions such as rand() and memory-related utilities.
5	using namespace std;	Allows usage of standard library names without writing std:: every time.
7	// Sequential Bubble Sort	A comment explaining that the next function performs sequential bubble sort.
8	void sequential_bubble_sort(const vector<int>& arr) {	Defines a function named sequential_bubble_sort. It takes a constant reference to a vector of integers.
9	vector<int> array = arr;	Creates a copy of the input array so the original data remains unchanged.
10	double start = omp_get_wtime();	Stores the starting execution time using OpenMP's wall-clock timer.
12	int size = array.size();	Calculates the number of elements in the vector.
13	for(int i = 0; i < size - 1; i++) {	Outer loop of bubble sort. Controls the number of passes.
14	for(int j = 0; j < size - i - 1; j++) {	Inner loop compares adjacent elements during each pass.
15	if(array[j] > array[j+1]) swap(array[j], array[j+1]);	Checks if adjacent elements are out of order. If true, swap them.
16	}	Ends the inner for-loop.
17	}	Ends the outer for-loop.
19	double end = omp_get_wtime();	Stores the ending execution time after sorting completes.
20	cout << "\n*** Sequential Bubble Sort ***\n";	Prints the heading for sequential bubble sort results.
21	cout << "Time Required: " << end - start << " seconds\n";	Calculates and prints total execution time.
22	}	Ends the sequential bubble sort function.
24	// Parallel Bubble Sort (Odd-Even Transposition Sort)	Comment describing the parallel sorting technique.
25	void parallel_bubble_sort(const vector<int>& arr) {	Defines the parallel bubble sort function.
26	vector<int> array = arr;	Copies the original array.
27	int size = array.size();	Stores array size.
28	double start = omp_get_wtime();	Starts timer for parallel execution.
30	for(int k = 0; k < size; k++) {	Controls sorting phases in odd-even transposition sort.
31	if(k % 2 == 0) {	Checks whether current phase is even.
32	#pragma omp parallel for schedule(static)	Creates parallel threads for the following loop using static scheduling.

33	for(int i = 1; i < size - 1; i += 2) {	Processes odd indexed pairs simultaneously.
34	if(array[i] > array[i+1]) swap(array[i], array[i+1]);	Swaps elements if they are unordered.
35	}	Ends odd-index loop.
36	} else {	Executes when current phase is odd.
37	#pragma omp parallel for schedule(static)	Parallelizes the even-index comparison loop.
38	for(int i = 0; i < size - 1; i += 2) {	Processes even indexed pairs simultaneously.
39	if(array[i] > array[i+1]) swap(array[i], array[i+1]);	Swaps unordered even-indexed adjacent values.
40	}	Ends even-index loop.
41	}	Ends else block.
42	}	Ends outer phase loop.
44	double end = omp_get_wtime();	Stores ending execution time.
45	cout << "\n*** Parallel Bubble Sort ***\n";	Prints heading for parallel bubble sort.
46	cout << "Time Required: " << end - start << " seconds\n";	Prints total execution time.
47	}	Ends parallel bubble sort function.
49	// Merge function	Comment introducing merge helper function.
50	void merge(vector<int>& array, int low, int mid, int high, vector<int> & temp)	Defines merge function used in merge sort.
51	int i = low, j = mid + 1, k = low;	Initializes pointers for left half, right half, and temp array.
52	while(i <= mid && j <= high) {	Runs while both halves contain elements.
53	if(array[i] <= array[j]) temp[k++] = array[i++];	Copies smaller element from left half into temp array.
54	else temp[k++] = array[j++];	Copies smaller element from right half into temp array.
55	}	Ends comparison loop.
56	while(i <= mid) temp[k++] = array[i++];	Copies remaining elements from left half.
57	while(j <= high) temp[k++] = array[j++];	Copies remaining elements from right half.
58	for(int idx = low; idx <= high; idx++) array[idx] = temp[idx];	Copies merged result back into original array.
59	}	Ends merge function.
61	// Sequential Merge Sort	Comment describing sequential merge sort.
62	void mergesort(vector<int>& array, int low, int high, vector<int> & temp)	Defines recursive merge sort function.
63	if(low < high) {	Checks whether array contains more than one element.
64	int mid = (low + high) / 2;	Finds middle index.
65	mergesort(array, low, mid, temp);	Recursively sorts left half.
66	mergesort(array, mid+1, high, temp);	Recursively sorts right half.
67	merge(array, low, mid, high, temp);	Merges sorted halves.
68	}	Ends if-condition.
69	}	Ends mergesort function.
71	void perform_merge_sort(const vector<int>& arr) {	Wrapper function to execute merge sort.

72	vector<int> array = arr;	Copies input array.
73	vector<int> temp(array.size());	Creates temporary array used during merging.
74	double start = omp_get_wtime();	Starts execution timer.
76	mergesort(array, 0, array.size() - 1, temp);	Calls merge sort on complete array.
78	double end = omp_get_wtime();	Stores ending time.
79	cout << "\n*** Sequential Merge Sort ***\n";	Prints heading.
80	cout << "Time Required: " << end - start << " seconds\n";	Prints total execution time.
81	}	Ends wrapper function.
83	// Parallel Merge Sort with OpenMP tasks	Comment describing parallel merge sort.
84	void p_mergesort(vector<int>& array, int low, int high, int depth) {	Defines recursive parallel merge sort.
85	if(low < high) {	Checks if recursion should continue.
86	int mid = (low + high) / 2;	Calculates middle index.
88	if(depth < 4) {	Limits task creation depth to avoid excessive threads.
89	#pragma omp task shared(array, temp)	Creates OpenMP task for left half sorting.
90	p_mergesort(array, low, mid, temp, depth + 1);	Recursively sorts left half in parallel.
92	#pragma omp task shared(array, temp)	Creates task for right half sorting.
93	p_mergesort(array, mid+1, high, temp, depth + 1);	Recursively sorts right half in parallel.
95	#pragma omp taskwait	Waits until all child tasks finish.
96	} else {	If maximum depth reached, continue sequential recursion.
97	p_mergesort(array, low, mid, temp, depth + 1);	Sequential recursion on left half.
98	p_mergesort(array, mid+1, high, temp, depth + 1);	Sequential recursion on right half.
99	}	Ends else block.
101	merge(array, low, mid, high, temp);	Merges sorted halves.
102	}	Ends if-condition.
103	}	Ends p_mergesort function.
105	void perform_parallel_merge_sort(const vector<int>& arr) {	Wrapper function for parallel merge sort.
106	vector<int> array = arr;	Copies input array.
107	vector<int> temp(array.size());	Creates temp array for merging.
108	double start = omp_get_wtime();	Starts timer.
110	#pragma omp parallel	Starts parallel region.
111	{	Begins parallel block.
112	#pragma omp single	Ensures only one thread starts recursive tasks.
113	p_mergesort(array, 0, array.size() - 1, temp);	Calls parallel merge sort.
114	}	Ends parallel block.
116	double end = omp_get_wtime();	Stores ending time.

117	cout << "\n*** Parallel Merge Sort ***\n";	Prints heading.
118	cout << "Time Required: " << end - start << " seconds\n";	Prints execution time.
119	}	Ends wrapper function.
121	int main() {	Main function where execution starts.
122	int SIZE;	Declares variable for array size.
123	int MAX = 1000;	Maximum random number value.
124	cout << "Enter size of array: ";	Prompts user to enter size.
125	cin >> SIZE;	Reads array size from user.
127	vector<int> array(SIZE);	Creates vector of given size.
128	for(int i = 0; i < SIZE; i++)	Loop for inputting array values.
129	{	Starts loop body.
130	//array[i] = rand() % MAX;	Commented line that could generate random numbers.
131	cin>>array[i];	Reads array element from user.
132	}	Ends input loop.
134	sequential_bubble_sort(array);	Calls sequential bubble sort.
135	parallel_bubble_sort(array);	Calls parallel bubble sort.
136	perform_merge_sort(array);	Calls sequential merge sort.
137	perform_parallel_merge_sort(array);	Calls parallel merge sort.
139	return 0;	Indicates successful program completion.
140	}	Ends main function.

Overall Concept:

The program compares sequential and parallel implementations of sorting algorithms. OpenMP is used to introduce parallelism using: **#pragma omp parallel for** for Bubble Sort **#pragma omp task** for Merge Sort Parallel programming can improve performance on multi-core processors because multiple operations execute simultaneously.